

Programación I

Clase 1 · Presentación de la materia y primeros pasos

UTN · Facultad Regional General Pacheco
2do cuatrimestre 2026

Hoja de ruta de hoy

Quiénes somos y qué vamos a hacer

Docentes, cursada, contenidos, aprobación

¿Qué es programar?

Actividad + algoritmo + entrada/proceso/salida

Corte

20 minutos

Elementos de programación

Variables, tipos de datos y operadores

Nuestro primer programa

Del enunciado al ejecutable, en vivo

Práctica guiada y cierre

Lo resuelven ustedes, con nosotros al lado

Los docentes



Angel Simón

Profesor

asimon@docentes.frgp.utn.edu.ar



Brian Lara

Profesor

blara@docentes.frgp.utn.edu.ar



Abel Faure

Jefe de Trabajos Prácticos

afaure@docentes.frgp.utn.edu.ar

Cómo cursamos

Encuentros presenciales

Martes 18:00 a 22:00 ▪ **Angel y Abel**

Viernes 18:00 a 22:00 ▪ **Brian y Abel**

Cómo nos comunicamos

Foro de avisos — lo importante se publica ahí

Foro de dudas generales — preguntá, respondemos todos

Mail — para cuestiones personales

Una advertencia honesta

Programar no se aprende mirando. Se aprende escribiendo código que no funciona y averiguando por qué. La materia está diseñada para que practiquen entre clase y clase: si sólo vienen a escuchar, no alcanza.

Qué vamos a aprender

El cuatrimestre completo, en siete pasos que se apilan uno sobre otro.

01 **Variables y constantes**
Guardar datos en memoria

02 **Operadores**
Hacer cuentas y comparaciones

03 **Estructura de secuencia**
Instrucciones una detrás de otra

04 **Estructura de decisión**
Simple y múltiple: el programa elige

05 **Estructura de repetición**
Ciclos exactos, inexactos y combinados

06 **Vectores**
Muchos datos bajo un mismo nombre

07 **Funciones**
Partir el problema en pedazos

★ **Trabajo Integrador Final**
Todo junto, resolviendo un problema real

Cómo se aprueba

Evaluación por procesos: hay actividades obligatorias durante toda la cursada.

75%

de asistencia

Como mínimo, a los encuentros presenciales.

1

Examen Parcial

Hay que realizarlo y aprobarlo.

TIF

Trabajo Integrador

Se entrega y se defiende oralmente.

El detalle completo está en el contrato didáctico, publicado en el campus. Léanlo esta semana: es el documento que rige la cursada.

Nuestras dos herramientas

Todo ejercicio de la materia se resuelve dos veces: primero se piensa, después se escribe.

1 Diagramación

Mediante diagramas de flujo, representar la solución a la lógica de la situación problemática. Es pensar la solución sin pelear con la sintaxis todavía.

2 Codificación

Mediante el lenguaje C++, escribir programas que efectivamente resuelvan el problema y se puedan ejecutar en una computadora.

Un diagrama bien pensado hace que codificar sea casi transcribir. Un diagrama saltado se paga en horas de errores.

Software

Hoy no instalamos nada: la instalación es la tarea de esta semana.

Hoy, en clase

CodeBlocks y Diagramas de Flujo

En casa, esta semana

Codificación: **CodeBlocks 25.03** (Windows y Linux)

Diagramación: **Draw.io**, papel y lápiz

Alternativas

Codificación	Visual Studio Code + plugin C++ (Windows, Linux, Mac) · Xcode (Mac)
--------------	---

Codificación	Zinjai (Windows) · Visual Studio Community (Windows)
--------------	--

Diagramación	Cualquier otro software para diagramar: Excalidraw
--------------	--

Campus Virtual

Foros

Avisos, Dudas generales, Cafetería

Contrato didáctico

Las reglas de la cursada

Cronograma

Qué se ve cada clase y cuándo es el parcial

Guía de actividades

Los ejercicios de toda la materia

Repositorio de archivos

Presentaciones y material de cada clase

Videotutoriales

Instalación y uso de las herramientas

Acceso

<https://frgp.cvg.utn.edu.ar/>

Usuario: **DNI . frgp**

Contraseña: **DNI**

Entrá esta semana y presentate en el foro Cafetería: es la forma más simple de verificar que tu usuario funciona.

¿Qué es programar?

Antes de escribir una sola línea de código, entendamos qué le estamos pidiendo a la máquina.

Actividad: Ayudarme a poner los anteojos de lectura

Sin computadoras. Yo, esta mochila y ustedes dándome instrucciones.

La consigna

En algún bolsillo de esta mochila hay un estuche celeste con unos anteojos. No me acuerdo en cuál. Ustedes me dictan instrucciones, una por vez, hasta que los tenga puestos. Ejecuto exactamente lo que dicen: ni una interpretación de más.

No interpreto

Si la orden admite dos lecturas, elijo la que quiera. Y suele ser la equivocada.

No completo

Si falta un paso, no lo agrego por mi cuenta. Simplemente no pasa.

No me salteo el orden

Ejecuto en el orden en que me lo dicen, aunque sea absurdo o imposible.

Acaban de escribir un algoritmo

Eso que fuimos armando entre todos tiene nombre propio.

Una secuencia finita, ordenada y precisa de pasos que resuelve un problema.

Preciso

Cada paso dice exactamente qué hacer. Nada queda librado a la interpretación.

Ordenado

El orden importa. Los mismos pasos en otro orden resuelven otra cosa, o nada.

Finito

Termina. Un procedimiento que nunca termina no resuelve el problema.

Repetible

Con los mismos datos de entrada da siempre el mismo resultado.

Programar es escribir algoritmos en un lenguaje que la computadora entiende.

Lo que inventaron sin saberlo

Cada cosa que necesitaron para la mochila es una unidad del programa de la materia.

“Abrí el bolsillo, después sacá el estuche”

Estructura de secuencia

“Si no son los de lectura, probá con otros anteojos”

Estructura de decisión

“Seguí abriendo bolsillos hasta encontrarlos”

Ciclo inexacto

“¿Cómo ponerse los anteojos? Agarralos, abrilos, ponelos detrás de las orejas...”

Función: un macropaso se divide en cinco micropasos

Todo esto lo necesitaron en quince minutos, sin computadora. El cuatrimestre es aprender a decírselo a una máquina.

Todo programa tiene la misma forma

Este esquema los va a acompañar todo el cuatrimestre. Empezar siempre por acá.

Entrada

Qué datos necesito

Los pide el usuario por teclado, o ya están en el programa.



Proceso

Qué hago con ellos

Cálculos, decisiones, repeticiones. La lógica de la solución.



Salida

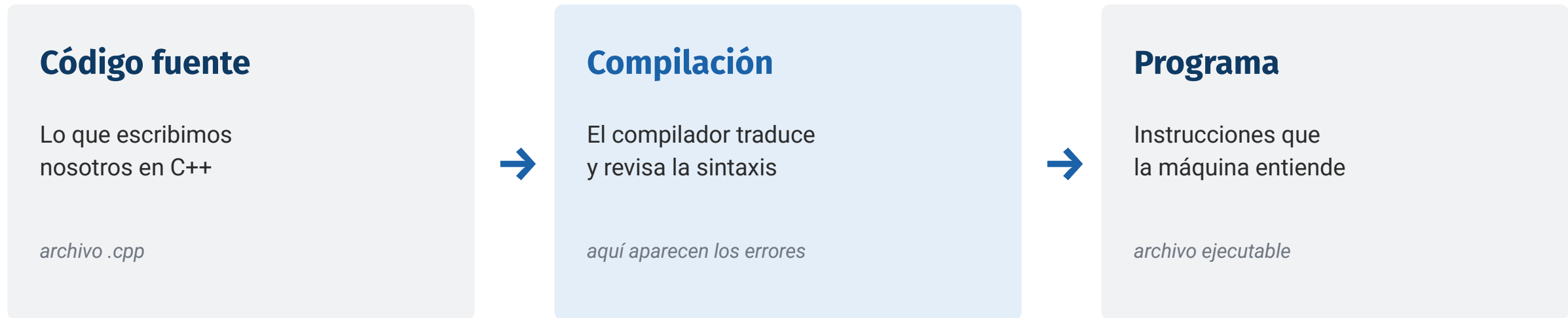
Qué informo

Lo que el programa muestra en pantalla como resultado.

Frente a cualquier enunciado, la primera pregunta no es "¿cómo lo programo?" sino "¿qué ingresa, qué hago y qué se muestra?".

Del código al programa

El proceso de transformar un **código fuente** en un **programa** que contiene las instrucciones comprensibles por la computadora se llama **compilación**.



Si el código tiene un error de sintaxis, la compilación falla y no hay programa. El compilador avisa qué línea no entendió: leer ese mensaje es parte del oficio.

El lenguaje tiene reglas estrictas

Cada elemento que usemos está sujeto a una sintaxis. Los elementos que C++ admite son:

1 Variables y constantes

2 Operadores

3 Expresiones

4 Palabras reservadas del lenguaje

Case-sensitive

Como muchos lenguajes, C y C++ hacen diferencia entre mayúsculas y minúsculas.

```
nombre  
Nombre  
NOMBRE
```

Son tres variables distintas.

Un punto y coma faltante, una llave sin cerrar o una mayúscula de más son motivos suficientes para que el programa no compile. No es maldad del compilador: es que no puede adivinar.

Corte

20 minutos.

Tipos de datos

Para representar cosas de la vida real —precios, nombres, edades— necesitamos distintos tipos.

TIPO	DESCRIPCIÓN	EJEMPLOS
<code>int</code>	Un número entero	42 -7 0
<code>float</code>	Un número con expresión decimal (real)	3.14 -0.5
<code>bool</code>	Un valor booleano: verdadero o falso	true false
<code>char</code>	Un carácter: letra, número o símbolo	'A' '7' '#'
<code>string</code>	Un texto	"Programación I"

Elegir bien el tipo importa: guardar un promedio en un `int` hace que 7.5 se convierta en 7.

Variables y constantes

Representación simbólica de espacios de memoria: es donde se almacenan los datos en procesamiento.

Variable

Se identifica con un tipo de dato y un identificador de nombre, ambos elegidos por el programador. Permite escribir un dato en memoria o leerlo. Su valor se puede modificar las veces que sea necesario.

```
int edad;  
edad = 20;  
edad = 21; // vale
```

Constante

Se identifica con la palabra reservada const, un tipo, un nombre y un valor que NO puede ser modificado durante el transcurso del programa en ejecución.

```
const int EDAD_MIN = 18;  
EDAD_MIN = 16;  
// error: no se puede
```

Convención: los nombres de constantes se escriben en MAYÚSCULAS. No es obligatorio para el compilador, pero sí para quien lea el código.

Cómo se declaran

Tipo, nombre y —opcionalmente— un valor inicial. Punto y coma al final, siempre.

Variables

```
int legajo;  
float sueldo;  
string nombre = "Programación I";  
char letra;
```

Constantes

```
const int EDAD_MIN = 18;  
const float IMPUESTO = 10.5;  
const char PAIS = 'A';
```

Elegir buenos nombres no es cosmética

Sin espacios, sin empezar con número, sin tildes y sin usar palabras reservadas. Pero además: que digan qué guardan. Dentro de tres semanas, **sueldoNeto** se entiende y **asd** no.

Operadores matemáticos

Los paréntesis funcionan igual que en matemática. En programación no se usan corchetes ni llaves para separar términos.

+ Suma

- Resta

***** Producto

/ División real o cociente entero

% Resto de la división entera

Antes de que lo cuente:

¿Qué da cada una de estas expresiones?

```
((2+3)*5)+10
```

```
2+3*5+10
```

```
5 / 2
```

```
5.0 / 2
```

```
5 % 2
```

Los resultados

Dos de estos sorprenden a casi todo el mundo la primera vez.

$$((2+3)*5)+10 \rightarrow 35$$

Los paréntesis mandan, igual que en matemática.

$$2+3*5+10 \rightarrow 27$$

Sin paréntesis, el producto se resuelve primero.

$$5 / 2 \rightarrow 2$$

Entero dividido entero da entero: la parte decimal se descarta.

$$5.0 / 2 \rightarrow 2.5$$

Basta con que uno sea real para que la división sea real.

$$5 \% 2 \rightarrow 1$$

El resto de dividir 5 por 2. Sirve muchísimo, ya van a ver.

La división entera es la fuente número uno de resultados raros en los primeros trabajos prácticos. Cuando un promedio les dé un número redondo sospechoso, miren acá.

El operador de asignación

Asigna la expresión que está a la derecha del operador en la variable que está a la izquierda.

```
edad = 50;  
latitud = 45.22;  
nombre = "Angel";  
caracter = 'X';  
a = b;  
precio = cant * pu;  
cont = cont + 1;
```

Error de sintaxis

50 = edad; 50 = 50;

A la izquierda tiene que haber una variable, no un valor.

Correcto pero sin sentido

edad = edad;

Compila sin quejarse. No hace absolutamente nada.

Mirá bien la última línea: cont = cont + 1

Como igualdad matemática es falsa: ningún número es igual a sí mismo más uno. Pero el = no es igualdad, es una orden: "tomá lo que hay en cont, sumale 1 y volvé a guardarlo en cont". Primero se resuelve la derecha, después se guarda a la izquierda.

Palabras reservadas

Tienen un significado especial para el lenguaje: no se pueden usar como nombres de variables.

Tipos de datos

`bool` `int` `float` `char` `short` `void` `long` `double`

Elementos de programación

`if` `else` `switch` `default` `break` `for` `while` `do` `return` `auto` `struct` `class`
`static` `virtual`

Operadores

`new` `delete` `sizeof`

Esta tabla es material de consulta: nadie la memoriza hoy. Queda publicada en el campus. Lo único que importa ahora es saber que existen, para no llamar `int` a una variable.

Nuestro primer programa

El mismo problema, tres veces: en castellano, en diagrama y en C++.

Ejercicio

Hacer un programa que permita ingresar dos números enteros por teclado. Luego calcular e informar la suma de ellos.

Antes de escribir una línea, contestemos las tres preguntas de siempre:

1

Datos de entrada

Dos números enteros.

2

Proceso

Operación matemática de suma de ambos números.

3

Información de salida

El resultado de la suma.

Resolución: instrucciones

Primero en castellano, después su traducción a C++.

01 Declarar las variables necesarias

```
int num1, num2, resultado;
```

02 Ingresar el primer número por teclado

```
cin >> num1;
```

03 Ingresar el segundo número por teclado

```
cin >> num2;
```

04 Realizar la suma

```
resultado = num1 + num2;
```

05 Mostrar por pantalla el resultado

```
cout << resultado;
```

Resolución: en diagrama

El mismo programa, dibujado. Cada instrucción tiene su forma, y la forma dice qué tipo de operación es.

Simbología



Entrada

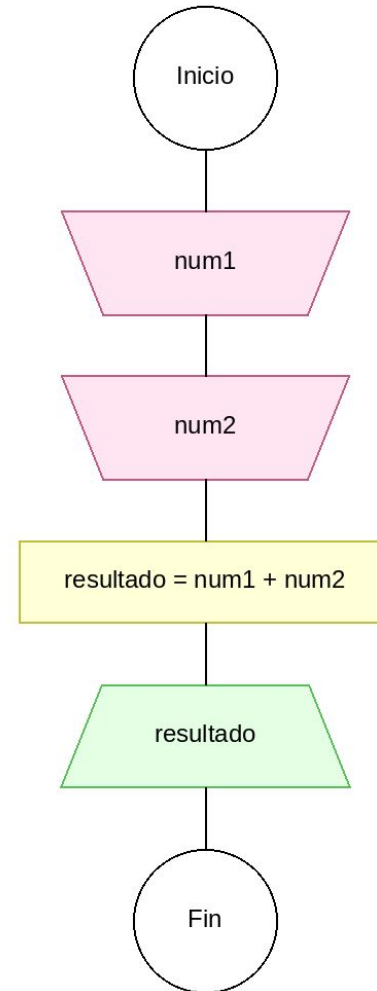


Salida



Proceso o asignación

La declaración de variables es opcional en el diagrama; en el código no lo es.



Resolución: código fuente

```
#include <iostream>
using namespace std;

int main(){
    int num1, num2, resultado;
    cin >> num1;
    cin >> num2;
    resultado = num1 + num2;
    cout << resultado;
    return 0;
}
```

#include <iostream>

Le pedimos al compilador la biblioteca de entrada y salida: sin ella no existen cin ni cout.

int main()

El punto donde empieza a ejecutarse el programa. Todo lo que va entre las llaves se ejecuta en orden.

cin >> · cout <<

Leer del teclado y escribir en pantalla. Las flechas apuntan hacia dónde va el dato.

return 0;

Le avisa al sistema operativo que el programa terminó bien.

Ahora en vivo

Lo escribimos desde cero, juntos. Nada de copiar y pegar.

01 Crear el proyecto o el archivo .cpp

Dónde se guarda y con qué nombre.

02 Escribir el código línea por línea

Notar que el editor colorea y autocompleta.

03 Compilar

Qué significa 0 errors, 0 warnings.

04 Romperlo a propósito

Le saco un punto y coma y volvemos a compilar. Leemos juntos el mensaje de error.

05 Ejecutar

Escribimos dos números y vemos el resultado.

06 Encontrar el ejecutable

Dónde quedó el archivo que la computadora corre.

Ahora ustedes: calorías quemadas

De a dos, con la computadora o en papel. Nosotros tres pasamos por los bancos.

Desarrollar un programa en C++ que solicite al usuario su nombre y la cantidad de kilómetros caminados. El programa debe calcular las calorías quemadas utilizando un valor fijo de 60 calorías por kilómetro y mostrar por pantalla un resumen con el nombre del usuario, el valor de la constante utilizada y el resultado del cálculo.

```
Ingrese su nombre: Martín  
Ingrese los kilómetros caminados: 7.5
```

```
≡ Resumen ≡  
Nombre: Martín  
Calorías por km: 60  
Kilómetros caminados: 7.5  
Calorías quemadas: 450
```

¿Qué entra?

¿Qué es constante?

¿Qué sale?

Ahora ustedes: calorías quemadas

De a dos, con la computadora o en papel. Nosotros tres pasamos por los bancos.

Desarrollar un programa en C++ que solicite al usuario su nombre y la cantidad de kilómetros caminados. El programa debe calcular las calorías quemadas utilizando un valor fijo de 60 calorías por kilómetro y mostrar por pantalla un resumen con el nombre del usuario, el valor de la constante utilizada y el resultado del cálculo.

```
Ingrese su nombre: Martín  
Ingrese los kilómetros caminados: 7.5
```

```
=== Resumen ===  
Nombre: Martín  
Calorías por km: 60  
Kilómetros caminados: 7.5  
Calorías quemadas: 450
```

¿Qué entra?

El nombre y los kilómetros. ¿De qué tipo es cada uno?

¿Qué es constante?

Las 60 calorías por km no cambian nunca. ¿Qué palabra reservada usamos?

¿Qué sale?

Cuatro líneas. Cuidado con la división y la multiplicación de reales.

Para el viernes

Cuatro cosas, ninguna larga. Hacerlas antes de la próxima clase importa.

01

Entrar al campus virtual

<https://frgp.cvg.utn.edu.ar/> · Usuario DNI.frgp · Contraseña DNI. Presentate en el foro Cafetería.

02

Leer el contrato didáctico

Está en el campus. Es el documento que rige la cursada: si algo no te cierra, preguntá el viernes.

03

Instalar CodeBlocks 25.03

Hay videotutoriales en el campus. Si se traba, escribí en el foro de dudas: no llegues sin instalarlo.

04

Terminar el ejercicio de calorías

Si no lo terminaste hoy, terminalo en casa. Traelo el viernes.

Nos vemos el viernes

asimon@docentes.frgp.utn.edu.ar · blara@docentes.frgp.utn.edu.ar · afaure@docentes.frgp.utn.edu.ar